

Composants qualifiant un autre composant

Besoin

Dans la plupart des cas, les colonnes des fichiers csv représentant des variables sont indépendantes les unes des autres ; la colonne “température” est considérée comme indépendante de la colonne “précipitations”. Certaines colonnes de contexte concernent l’ensemble des colonnes de variables ; c’est le cas par exemple des colonnes “site” et “date”. Néanmoins, Il peut y avoir une logique métier à ce qu’une ou plusieurs colonnes ne concernent qu’une autre colonne et pas l’ensemble des colonnes d’un même fichier csv. Par exemple, l’écart-type d’une valeur moyenne d’une variable, le nombre de valeurs utilisées pour le calcul ou plus basiquement, la méthode et l’unité d’une valeur d’une variable. On parle alors de colonnes qualifiant une colonne ou de composant(s) qualifiant un composant pour la définition du fichier de configuration d’un SI. L’objectif en utilisant cette option est de permettre à l’utilisateur des données de comprendre qu’une valeur d’une information est associée à la valeur d’une autre information (ex: la valeur d’un écart type vaut pour telle valeur de telle variable). Il faut noter qu’au sein d’un même fichier, ces valeurs associées à d’autres valeurs peuvent être constantes ou bien varier selon les valeurs du fichier. Par exemple, dans un fichier de données csv, pour la variable “temperature”, l’unité et la méthode peuvent être les mêmes pour toutes les valeurs de température alors que l’écart-type varie très probablement entre les valeurs de température (une moyenne ici).

Cas d’usage

Plusieurs cas de figure ont été identifiés et sont détaillés ci-dessous sur la base d’exemples concrets.

Fichier “horizontal” avec des noms de colonnes uniques :

Ex: | Site | Date | var_a | ecart-type_var_a | unité_var_a | var_b | ecart-type_var_b
| unité_var_b | |———|———|———|———|———|———|———|
-| | Paris | 2022-12-03 | 12.3 | 0.25 | cm | 125.14 | 3.95 | ml | | Paris | 2020-11-28 | 21.8 | 1.54 |
cm | 98.21 | 3.95 | ml |

Stockage sans verticalisation

Si le nombre de colonnes est figé, le fichier peut être stocké sans verticalisation même si ce n’est pas la recommandation. En effet, si tel est le cas, il ne sera pas possible de disposer du niveau variable pour positionner des droits. En effet, le droit est positionnable pour une ligne. Et une ligne enregistrée en base correspondra à l’ensemble des valeurs d’une ligne du fichier pour les colonnes var_a, ecart-type_var_a, unité_var_a, var_b, ecart-type_var_b, unité_var_b en plus des colonnes site et date. De plus, les variables et leurs qualifieurs ne pourront pas être

associés directement à un référentiel où l'on décrit les variables en fournissant notamment un uri pour préparer l'interopérabilité sémantique.

Si on considère dans notre exemple qu'au sein d'un fichier csv de ce type de données, il n'y a qu'une unité possible pour chaque variable (l'unité peut changer entre deux fichiers csv de ce type de données), la forme du fichier suivant est à considérer :

	Unité	cm		ml	
Site	Date	var_a	ecart-type_var_a	var_b	ecart-type_var_b
Paris	2022-12-03	12.3	0.25	125.14	3.95
Paris	2020-11-28	21.8	1.54	98.21	3.95

Le composant OA_constantComponents et le bon usage du tagg **ORDER_X** permet dans ce cas de traiter les valeurs des unités (les valeurs des unités sont répétées en base comme dans le cas du premier exemple de format de fichier csv).

Le format du fichier de sortie sera dans les deux cas, équivalent au premier format de fichier d'échange présenté.

////////////////////////////////////
**PARTIE QUE NOUS N'AVONS PAS DISCUTE TOUS ENSEMBLE MAIS
 IMPORTANTE DES MAINTENANT, A VALIDER OU PAS AVEC PHILIPPE
 SELON EFFORT DE DEVELOPPEMENT ASSOCIE**

Question en suspens : y a t-il un intérêt à distinguer ces colonnes écart-type et unité qui n'ont pas de sens si on ne fournit pas a minima la variable et/ ou la variable et sa valeur ? -> propositions : pas véritablement à ce stade, mais on pourrait souhaiter à termes quand l'ihm d'extraction le permettra faire en sorte que ces composants ne puissent pas être extraits sans le composant auquel ils sont nécessairement associés.

Si on retenait dès à présent qu'il faut les distinguer des composants indépendants, il faudrait pouvoir déclarer dans le yaml une section permettant d'aboutir au format de stockage json en base proposé plus loin soit :

```
{variable: var_a
  { __VALUE__: 12.3  #réservé, aucun autre composant ne peut s'appeler __VALUE__
    variable: a
    ecart-type_var_a: 0.25
    unité_var_a : cm
  }
}
```

A noter ici que l'on ne peut pas fournir non plus la valeur de la variable sans les composants qui la qualifient (écart-type et unité).

Une section `OA_qualifierComponentForComponent` pourrait permettre de déclarer un composant comme "adjacent" (assimilé à "qualifiant") de n'importe quel autre composant au sein d'un même type de données.

Ex sur la base du deuxième format de fichier csv proposé ci-dessus:

```
OA_basicComponents:
  dty_site:
    OA_importHeader: "Site"
    OA_tags: [__ORDER_1__]
  dty_date:
    OA_importHeader: "Date"
    OA_tags: [__ORDER_2__]
  dty_var_a:
    OA_importHeader: "var_a"
    OA_tags: [__ORDER_3__]
  dty_sd_var_a:
    OA_importHeader: "ecart-type_var_a"
    OA_tags: [__ORDER_4__]
  dty_var_b:
    OA_importHeader: "var_b"
    OA_tags: [__ORDER_6__]
  dty_sd_var_b:
    OA_importHeader: "ecart-type_var_b"
    OA_tags: [__ORDER_7__]
OA_constantComponents:
  dty_unit_var_a:
    OA_exportHeader:
      OA_title:
        fr: "unité pour la variable a"
        en: "unit for variable a"
    OA_required: true
    OA_importHeaderTarget:
      OA_rowNumber: 1
      OA_columnNumber: 3
    OA_qualifierComponentForComponent: dty_var_a
    OA_tags: [__ORDER_5__]
  dty_unit_var_b:
    OA_exportHeader:
      OA_title:
        fr: "unité pour la variable b"
```

```

        en: "unit for variable b"
    OA_required: true
    OA_importHeaderTarget:
        OA_rowNumber: 1
        OA_columnNumber: 5
    OA_qualifierComponentForComponent: dty_var_a
    OA_tags: [__ORDER_8__]

```

OA_qualifierComponentForComponent pourrait être utilisé dans les composants basiques, calculés et les constantes.

FIN DE LA PARTIE QUE NOUS N'AVONS PAS DISCUTE TOUS ENSEMBLE MAIS IMPORTANTE DES MAINTENANT, A VALIDER OU PAS AVEC PHILIPPE SELON EFFORT DE DEVELOPPEMENT ASSOCIE

Stockage avec verticalisation

En revanche, si le nombre de colonnes (donc de variables) n'est pas figé à l'avance, le fichier ne peut pas être stocké sans l'usage de la verticalisation ; le composant OA_patternCompents est alors incontournable.

Sur la base du premier fichier csv d'exemple ci-dessus, on aboutirait à un stockage de 12 lignes dans la bd comme ci-dessous :

```

{
  site: Paris
  date: 2022-12-03
  variable: var_a
  value: 12.3
}
{
  site: Paris
  date: 2022-12-03
  variable: ecart-type_var_a
  value: 0.25
}
{
  site: Paris
  date: 2022-12-03
  variable: unité_var_a
  value: cm
}

```

```

}
{
  ...
}

```

avec un fichier de sortie de la forme suivante (pour la première ligne):

Site	Date	variable	valeur
Paris	2022-12-03	var_a	12.3
Paris	2022-12-03	ecart-type_var_a	1.54
Paris	2022-12-03	unité_var_a	cm
Paris	2022-12-03	var_b	125.14
Paris	2022-12-03	ecart-type_var_b	3.95
Paris	2022-12-03	unité_var_b	ml

Problème : on a une colonne variable et deux colonnes sd et unité qui ne sont pas à proprement parler des variables. sd et unité ne sont rattachées à la bonne variable que grâce à leur nom incluant var_a (en + des valeurs de Site et Date)

Solution : pouvoir isoler la variable ou la valeur en tant que composant incluant d'autres composants tels que les colonnes adjacentes sd et unité avec un stockage de ce type :

```

{variable : var_a
  { __VALUE__: 12.3  #réservé, aucun autre composant ne peut s'appeler __VALUE__
    variable: a
    ecart-type_var_a: 0.25
    unité_var_a : cm
  }
}

```

avec un fichier de sortie de la forme suivante (pour la première ligne):

Site	Date	variable	valeur	ecart-type	unité
Paris	2022-12-03	var_a	12.3	1.54	cm
Paris	2022-12-03	var_b	125.14	3.95	ml

La configuration peut être de la forme suivante en utilisant une section OA_adjacentComponentQualifiers:

```

OA_patternComponents:
  variable_value:
    OA_patternForComponents: "(.*)"
    OA_exportHeader:
      OA_title:
        fr: "valeur"
        en: "value"
    OA_tags: [__ORDER_2__]
    OA_required: false #la valeur peut ne pas être renseignée
    OA_checker:
      OA_name: OA_float
    OA_componentQualifiers: # au pluriel car on peut recuperer plusieurs informations
      - variable:
          OA_exportHeader:
            OA_i18n:
              fr: "variable"
              en: "variable"
            OA_tags: [__ORDER_1__]
            OA_required: true
            OA_checker:
              OA_name: OA_reference
              OA_params:
                OA_reference:
                  OA_name: tr_variable_var
    OA_adjacentComponentQualifiers:
      - sd:
          OA_importHeaderPattern: "ecart-type_($0)" #ici $0 vaut "var_a"
          #ou "ecart-type_var_{$1} a_b_c ->(.)_(.)_(.) → $0 = a_b_c $1 = a, $2=b, $3=
          # recherche d'une colonne "ecart-type..." à droite (par convention) de la prem
          OA_required: false # la valeur n'est pas obligatoire
          OA_mandatory: false # la colonne ecart-type_var_x n'est pas obligatoire.
          OA_tags: [__ORDER_3__]
          OA_exportHeader:
            OA_i18n:
              fr: "écart type"
              en: "standard deviation"
            OA_checker:
              OA_name: OA_float
      - OA_name:
          OA_importHeaderPattern: "unité_($0)" # recherche d'une colonne "unité..." à dr
          OA_required: false
          OA_mandatory: false

```

```

OA_tags: [__ORDER_4__]
OA_exportHeader:
  OA_i18n:
    fr: "unité"
    en: "unit"
OA_checker:
  OA_name: OA_string

```

//////////////////////////////////// CAS NON TRAITE MAIS ON GARDE POUR TRACE

On peut vouloir s'appuyer sur les noms des colonnes plutôt que sur leurs positions relatives. En effet, dans le cas où l'ordre des colonnes n'est pas conforme à celui (assez logique) illustré dans l'exemple précédent, seul le nom des colonnes peut permettre de faire le job comme avec l'exemple ci-dessous :

Site	Date	var_a	var_b	ecart- type_var_a	unité_var_a	ecart- atype_var_b	unité_var_b
Paris	2022-12-03	12.3	125.14	0.25	cm	3.89	ml
Paris	2020-11-28	21.8	98.21	1.54	cm	2.54	ml

Pour avoir le même stockage en base et la même sortie qu'avec l'exemple précédent, on pourrait avoir une configuration de ce type :

```

OA_patternComponents:
  variable_value:
    OA_patternForComponents: "(^(?!(ecart-type_|unité_)).*)" # toutes les en-têtes n'ayant pas ces préfixes
    OA_exportHeader:
      OA_title:
        fr: "valeur"
        en: "value"
    OA_tags: [__ORDER_2__]
    OA_required: false
    OA_checker:
      OA_name: OA_float
    OA_componentQualifiers: # au pluriel car on peut recuperer plusieurs informations
                           # --> doit-on en identifier un en main et d'autres en adjointes
      - variable:
        OA_exportHeader:

```

```

    OA_i18n:
      fr: "variable"
      en: "variable"
    OA_tags: [__ORDER_1__]
    OA_required: true
    OA_checker:
      OA_name: OA_reference
      OA_params:
        OA_reference:
          OA_name: tr_variable_var
OA_adjacentComponentQualifiers:
- sd :
  OA_importHeaderPattern: "ecart-type_$0" # recherche d'une colonne "ecart-type_"
  OA_required: false # valeur requise ?
  OA_mandatory: false # colonne requise ?
  OA_tags: [__ORDER_3__]
  OA_exportHeader:
    OA_i18n:
      fr: "écart type"
      en: "standard deviation"
    OA_checker:
      OA_name: OA_float
- OA_name:
  OA_importHeaderPattern: "unité_$0" # recherche d'une colonne "unité_var_x" n'importe
  OA_required: false
  OA_mandatory: false
  OA_tags: [__ORDER_4__]
  OA_exportHeader:
    OA_i18n:
      fr: "unité"
      en: "unit"
    OA_checker:
      OA_name: OA_string

```

On doit pouvoir obtenir cette présentation en sortie (pour la première ligne):

Site	Date	variable	valeur	ecart-type	unité
Paris	2022-12-03	var_a	12.3	0.25	cm
Paris	2022-12-03	var_b	125.14	3.89	ml

FIN DU CAS NON TRAITE MAIS GARDE POUR TRACE //////////////////////////////////////

Fichier “horizontal” avec des noms de colonnes non uniques :

Ex: | Site | Date | var_a | ecart-type | unité | var_b | ecart-type | unité | |——-|—————|——
——-|—————|——-|——-|—————|——-| | Paris | 2022-12-03 | 12.3 | 0.25 | cm | 125.14 | 3.95
| ml | | Paris | 2020-11-28 | 21.8 | 1.54 | cm | 98.21 | 3.95 | ml |

Le fait d’avoir des colonnes avec des mêmes noms d’en-tête ne permet pas l’utilisation d’un composant basique, une erreur devrait être levée dès la validation du yaml (à tester). Le recours à la verticalisation via OA_patternComponents est a priori possible mais donnerait in fine quelque chose comme ça :

```
{
  site: Paris
  date: 2022-12-03
  variable: var_a
  value: 12.3
}
{
  site: Paris
  date: 2022-12-03
  variable: ecart-type
  value: 0.25
}
{
  site: Paris
  date: 2022-12-03
  variable: unité
  value: cm
}
{
  ...
}
```

avec un fichier de sortie de la forme suivante (pour la première ligne):

Site	Date	variable	valeur
Paris	2022-12-03	var_a	12.3
Paris	2022-12-03	ecart-type	1.54
Paris	2022-12-03	unité	cm
Paris	2022-12-03	var_b	125.14
Paris	2022-12-03	ecart-type	3.95
Paris	2022-12-03	unité	ml

créant une impossibilité de savoir quelle valeur d'écart type ou d'unité associer à une valeur d'une variable. Selon la clé naturelle, il est possible que l'insertion de données ne passe même pas (cas si clé sur site + date + variable dans l'exemple).

L'utilisation de composants dans un composant via `OA_adjacentComponentQualifiers` permet d'échapper ce cas en respectant la clé naturelle : les composants adjacents étant associés au composant de la clé naturelle + l'identificateur du `OA_patternComponent` (ici "variable_value").

La configuration est très proche de celle présentée ci-avant, avec la forme suivante :

```
OA_patternComponents:
  variable_value:
    OA_patternForComponents: "(.*)"
    OA_exportHeader:
      OA_title:
        fr: "valeur"
        en: "value"
    OA_tags: [__ORDER_2__]
    OA_required: false
    OA_checker:
      OA_name: OA_float
    OA_componentQualifiers:      # au pluriel car on peut recuperer plusieurs informations
                                # --> doit-on en identifier un en main et d'autres en adjac
      - variable:
        OA_exportHeader:
          OA_i18n:
            fr: "variable"
            en: "variable"
        OA_tags: [__ORDER_1__]
        OA_required: true
        OA_checker:
          OA_name: OA_reference
          OA_params:
            OA_reference:
              OA_name: tr_variable_var
    OA_adjacentComponentQualifiers:
      - sd:
        OA_importHeaderPattern: "ecart-type" # recherche d'une colonne "ecart-type" à
        OA_required: false # valeur requise ?
        OA_mandatory: false # colonne requise ?
        OA_tags: [__ORDER_3__]
        OA_exportHeader:
          OA_i18n:
```

```

        fr: "écart type"
        en: "standard deviation"
    OA_checker:
        OA_name: OA_float
- unit:
    OA_importHeaderPattern: "unite" # # recherche d'une colonne "unité" à droite (}
    OA_required: false
    OA_mandatory: false
    OA_tags: [__ORDER_4__]
    OA_exportHeader:
        OA_i18n:
            fr: "unité"
            en: "unit"
    OA_checker:
        OA_name: OA_string

```

Avec un fichier de sortie équivalent au cas précédent, de la forme suivante (pour la première ligne):

Site	Date	variable	valeur	ecart-type	unité
Paris	2022-12-03	var_a	12.3	1.54	cm
Paris	2022-12-03	var_b	125.14	3.95	ml

Note : les correspondances (ou tokens: \$0, \$1, \$2, ...) trouvées dans les en-têtes de colonnes selon le pattern peuvent potentiellement être utilisées dans les sections OA_exportHeader.

Note: Tous les composants identifiés dans un OA_patternComponents sont considérés d'emblée comme des composants adjacents au sens où ils sont tous stockés dans un tableau de clés/valeurs (ex pour SWC_1_10 avec les colonnes écart-type et unités-> le nom de la variable, sa valeur, son unité, sa profondeur, son numéro de répétition sont stockées dans un même tableau).

Verticalisation incluant des constantes comme composants adjacents

L'exemple ci-dessous, déjà présenté dans la section sans verticalisation est également à considérer quand on verticalise.

	Unité	cm		ml	
	Méthode	diffraction		sublimation	
Site	Date	var_a	ecart-type	var_b	ecart-type

	Unité	cm		ml	
Paris	2022-12-03	12.3	0.25	125.14	3.95
Paris	2020-11-28	21.8	1.54	98.21	3.95

Il correspond en effet à des cas fréquents de formats de fichiers utilisés quand des constantes sont associées à des colonnes de valeurs de variables. On peut gérer un en-tête contenant plusieurs informations sur des lignes distinctes plutôt que d'avoir ces informations répétées dans des colonnes à côté de la colonne des valeurs de la variable.

Le fichier de sortie conforme au stockage en base serait de la forme suivante

Site	Date	variable	valeur	ecart-type	unité	méthode
Paris	2022-12-03	var_a	12.3	1.54	cm	diffraction
Paris	2022-12-03	var_b	125.14	3.95	ml	sublimation

Il est proposé d'utiliser `OA_importHeaderTarget` dans la section `OA_adjacentComponentQualifiers` pour récupérer ces constantes propres à une variable comme illustré ci-dessous pour les méthodes et les unités.

```
OA_patternComponents:
  variable_value:
    OA_patternForComponents: "(.*)"
    OA_exportHeader:
      OA_title:
        fr: "valeur"
        en: "value"
    OA_tags: [__ORDER_4__]
    OA_required: false
    OA_checker:
      OA_name: OA_float
    OA_componentQualifiers: # au pluriel car on peut recuperer plusieurs informations o
                          # --> doit-on en identifier un en main et d'autres en adjac
      - variable:
        OA_exportHeader:
          OA_i18n:
            fr: "variable"
            en: "variable"
        OA_tags: [__ORDER_3__]
        OA_required: true
        OA_checker:
```

```

        OA_name: OA_reference
        OA_params:
            OA_reference:
                OA_name: tr_variable_var
OA_adjacentComponentQualifiers:
- sd:
    OA_importHeaderPattern: "ecart-type" # recherche d'une colonne "ecart-type" à
    OA_required: false # valeur requise ?
    OA_mandatory: false # colonne requise ?
    OA_tags: [__ORDER_5__]
    OA_exportHeader:
        OA_i18n:
            fr: "écart type"
            en: "standard deviation"
    OA_checker:
        OA_name: OA_float
- unit:
    OA_importHeaderTarget:
        OA_rowNumber: 1
    OA_required: false # valeur requise ?
    OA_mandatory: false # colonne requise ?
    OA_tags: [__ORDER_6__]
    OA_exportHeader:
        OA_i18n:
            fr: "unité"
            en: "unit"
- method:
    OA_importHeaderTarget:
        OA_rowNumber: 2
    OA_required: false
    OA_mandatory: false
    OA_tags: [__ORDER_7__]
    OA_exportHeader:
        OA_i18n:
            fr: "méthode"
            en: "method"
    OA_checker:
        OA_name: OA_string

```

Possibilité d'avoir des composants qualifiants calculés

pas prioritaire et à revisiter ensemble ultérieurement, à moins que des cas d'usage soient identifiés

Ex proposé par Philippe :

```
OA_computedComponentQualifiers:
- OA_name: correctedCO2Value
  OA_computation:
    OA_expression: "datum.co2_value / (1 - datum.co2_value.humidity)"
  OA_exportHeader:
    OA_title:
      fr: "valeur corrigée de CO2"
      en: "corrected CO2 value"
  OA_tags: [__ORDER_8__]
- OA_name: convertedValue
  OA_computation:
    OA_expression: "datum.co2_value * datum.co2_conversion_factor"
  OA_exportHeader:
    OA_title:
      fr: "valeur convertie"
      en: "converted value"
  OA_tags: [__ORDER_9__]
```